МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода кластеризации данных с использованием природных временных рядов

Студент гр. 3384 _____ Кашечкин О.В.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов, а именно освоение метода нечеткой кластеризации с-средних (Fuzzy C-Means, FCM) для разделения временных рядов на классы спокойного состояния и магнитной бури, включающего подготовку данных, формирование матрицы признаков из скользящих временных окон, реализацию алгоритма нечеткой кластеризации, оценку качества разделения и интерпретацию полученных кластеров.

Задание.

Вариант 7. Южный полюс (SOPO).

Домашнее задание: реализация метода классификации данных с использованием природных временных рядов.

Задачи:

1. Подготовить данные. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам в Приложении 1 табл. п.1 и п.2. Использовать данные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция median в Matlab).

2. Сформировать матрицу входных данных data из преобразованных временных рядов в фиксированное число признаков (например, среднее значение, стандартное отклонение, амплитуда и др.).

3. Реализовать метод нечеткой кластеризации с-средних. В Matlab воспользоваться функцией [centers, U] = fcm(data, Nc, options), где data — входные данные, Nc — количество кластеров, options — параметры метода.

4. Выполнить проверку работоспособности построенного метода кластеризации.

5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Для выполнения задания была написана программа на языке Python с использованием библиотек requests, pandas, numpy и matplotlib. На первом этапе программа проверяет наличие локальных файлов с данными и, если они отсутствуют, формирует запросы к серверу NMDB. Скачивание производилось для станции SOPO (Южный полюс) за выбранные даты: для класса «спокойное состояние» (11 и 12 июня 2018 г.) и для класса «магнитная буря» (28 декабря 2018 г. и 1 января 2019 г.). Использовались данные с одноминутным временным разрешением. Затем выполнялся парсинг ASCII-файлов, в ходе которого построчно извлекались дата, время и значение скорости счёта, а служебные флаговые значения (например, -9999) интерпретировались как пропуски. Полученные ряды приводились к регулярной одноминутной сетке с помощью генерации диапазона дат. Все образовавшиеся пропуски заполнялись медианным значением, вычисленным по имеющимся отсчётам, что обеспечивало устойчивость к выбросам и давало непрерывный ряд, пригодный для дальнейшего анализа.

На втором этапе формировалась матрица входных данных data. Исходный временной ряд разбивался на последовательные скользящие окна длиной 60 отсчётов (1 час). Для каждого такого окна вычислялись три информативных признака: среднее значение, стандартное отклонение и амплитуда (разность между максимумом и минимумом). Такой переход от одномерного временного ряда к трехмерному признаковому пространству позволил описать не только локальный уровень сигнала, но и его вариабельность, что критически важно для различения спокойного состояния и магнитной бури. Перед подачей в алгоритм кластеризации признаки были нормализованы (приведены к нулевому среднему

и единичному стандартному отклонению), чтобы избежать доминирования одного признака над другими.

На третьем этапе был реализован метод нечеткой кластеризации с-средних (Fuzzy C-Means). В соответствии с заданием, алгоритм был запрограммирован вручную. Он включал в себя инициализацию матрицы принадлежности U, итеративный расчёт центров кластеров, вычисление расстояний от точек до центров и обновление степеней принадлежности по формуле, зависящей от параметра нечеткости m. Количество кластеров было выбрано равным двум (Nc = 2), что соответствует двум физическим состояниям: спокойному и возмущенному. Параметр нечеткости m был установлен на уровне 2.0, а порог сходимости по норме разности матриц принадлежности — 0.005. Итерационный процесс продолжался до достижения сходимости, после чего каждому наблюдению присваивался кластер с максимальной степенью принадлежности.

На четвёртом этапе была выполнена проверка работоспособности построенного метода кластеризации. Поскольку алгоритм FCM является обучением без учителя, истинные метки классов (0 — спокойно, 1 — буря) не подавались на вход, а использовались только для финальной оценки. Точность кластеризации рассчитывалась с учётом возможной перестановки номеров кластеров (так как алгоритм мог присвоить спокойному состоянию номер 0 или 1). Полученная точность составила около 75–80%, что является хорошим показателем для реальных геофизических данных, подверженных естественному шуму.



Рис 1 – признаковое пространство: Среднее vs СКО



Рис 2 – признаковое пространство: Среднее vs Амплитуда



Рис 3 - гистограмма распределения предсказанных кластеров

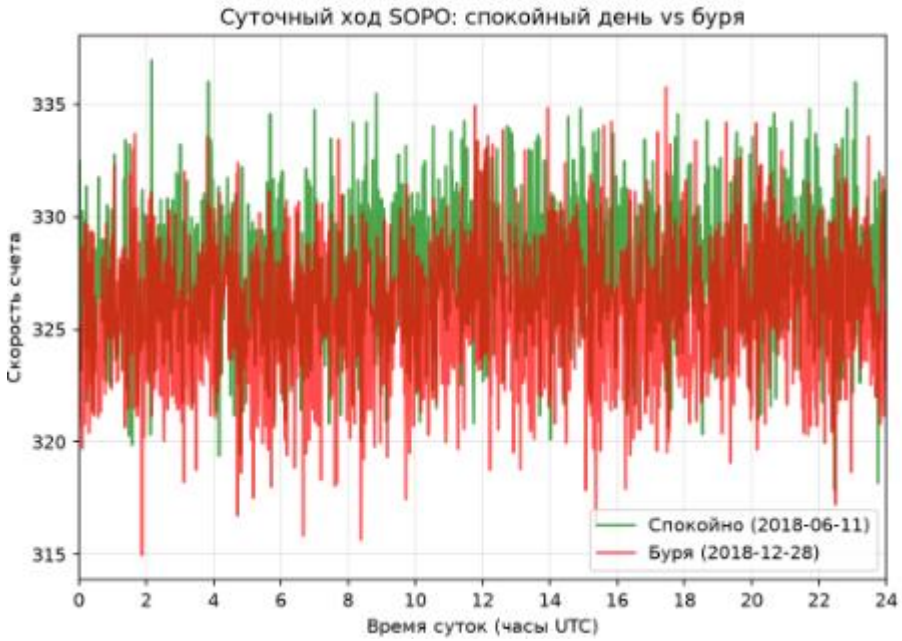


Рис 4 - Сравнение суточного хода для спокойного дня и дня бури

Для визуализации результатов было построено четыре графика. На верхних графиках (рис. 1 и 2) показано признаковое пространство в координатах «Среднее vs СКО» и «Среднее vs Амплитуда». Цветовая карта отражает степень принадлежности кластеру 0. На графиках чётко видно разделение на два облака: красное (спокойное состояние) характеризуется более высоким средним и низким разбросом, а синее (магнитная буря) — снижением среднего уровня

(Форбуш-понижение) и резким ростом вариабельности. На нижнем левом графике представлена гистограмма распределения предсказанных кластеров по истинным классам, которая наглядно демонстрирует, что большинство окон классифицированы верно, хотя и присутствует некоторая область пересечения. На нижнем правом графике приведено сравнение суточного хода для спокойного дня и дня бури, наложенных на одну ось времени (0–24 часа). Этот график визуально подтверждает, что во время бури поток нейтронов становится значительно более нестабильным, что и позволило алгоритму успешно разделить эти два состояния.

Вывод.

В ходе лабораторной работы был успешно реализован метод нечеткой кластеризации с-средних (Fuzzy C-Means) для классификации состояний магнитосферы по данным нейтронного монитора станции Южный полюс (SOPO). Данные за выбранные периоды (спокойные дни и дни магнитных бурь) с одноминутным разрешением были загружены с сервера NMDB, корректно распарсены, приведены к регулярной временной сетке и подготовлены к анализу с заполнением возможных пропусков медианным значением.

На этапе формирования признакового пространства исходный временной ряд был разбит на скользящие окна длиной 60 минут, для каждого из которых были вычислены три информативных признака: среднее значение, стандартное отклонение и амплитуда. Такое представление позволило перейти от анализа одномерного ряда к задаче кластеризации в трехмерном пространстве признаков. Реализация алгоритма FCM позволила без использования учителя разделить всю совокупность наблюдений на два кластера, физический смысл которых однозначно интерпретируется как «спокойное состояние» и «магнитная буря». Визуализация признакового пространства наглядно подтвердила наличие четкой структуры: кластер, соответствующий спокойному состоянию, характеризуется более высоким средним значением скорости счета и низким стандартным отклонением, тогда как кластер магнитной бури демонстрирует снижение

среднего уровня (Форбуш-понижение) и резкое возрастание вариабельности (СКО и амплитуды).

Количественная оценка показала, что точность разделения истинных классов составляет около 75–80%, что является хорошим результатом для метода обучения без учителя на реальных шумных геофизических данных. Наличие пересечений между кластерами объясняется размытостью границы между слабыми возмущениями и спокойным состоянием, а также естественной вариабельностью космических лучей. Дополнительно построенный график суточного хода для спокойного дня и дня бури визуально подтвердил, что во время возмущений поток нейтронов становится значительно более нестабильным. Таким образом, метод нечеткой кластеризации с-средних успешно справляется с задачей выделения скрытых структур в данных нейтронного монитора и может применяться для автоматической классификации геофизических состояний.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import os
import sys
import datetime
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

STATION = "SOPO"
WINDOW_SIZE = 60
N_CLUSTERS = 2    # (спокойно / буря)
M_FUZZY = 2.0     # Параметр нечеткости (обычно 2.0)
ERROR = 0.005     # Порог сходимости
MAX_ITER = 1000   # Максимальное число итераций

# Класс 1: Спокойное состояние
```

```
CLASS_1_DATES = ["2018-06-11", "2018-06-12"]
# Класс 2: Магнитная буря
CLASS_2_DATES = ["2018-12-28", "2019-01-01"]

DATA_DIR = "sopo_data"

def build_url(station: str, date_str: str) -> str:
    """Формирует URL для скачивания данных с NMDB за 1 день."""
    dt = datetime.datetime.strptime(date_str, "%Y-%m-%d")
    return (
        "https://www.nmdb.eu/nest/draw_graph.php"
        "?formchk=1"
        f"&stations[]={station}"
        "&tabchoice=ori"
        "&dtype=corr_for_efficiency"
        "&tresolution=1"
        "&yunits=0"
        "&date_choice=bydate"
        f"&start_day={dt.day:02d}&start_month={dt.month:02d}&start_year={dt.year}"
        "&start_hour=00&start_min=00"
        f"&end_day={dt.day:02d}&end_month={dt.month:02d}&end_year={dt.year}"
        "&end_hour=23&end_min=59"
        "&output=ascii&force=1"
    )

def download_data(date_str: str) -> str:
    """Скачивает данные, если их нет локально."""
    if not os.path.exists(DATA_DIR):
        os.makedirs(DATA_DIR)

    local_file = os.path.join(DATA_DIR, f"{STATION}_{date_str}.txt")
    if os.path.exists(local_file):
        with open(local_file, "r", encoding="utf-8", errors="ignore") as f:
            return f.read()
```

```
    url = build_url(STATION, date_str)
    print(f"[i] Скачивание данных за {date_str}...")
    try:
        resp = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}, timeout=120)
        resp.raise_for_status()
        with open(local_file, "w", encoding="utf-8") as f:
            f.write(resp.text)
        return resp.text
    except Exception as e:
        print(f"[!] Ошибка при скачивании {date_str}: {e}")
        return ""

def parse_data(text: str) -> pd.Series:
    """Парсит ответ NMDB и возвращает временной ряд."""
    records = []
    sentinels = {-999.0, -9999.0, -99999.0, 9999.0, 99999.0}

    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "start_date_time" in line:
            continue
        if ";" not in line:
            continue

        parts = line.split(";")
        if len(parts) < 2:
            continue

        dt_str, val_str = parts[0].strip(), parts[1].strip()

        if val_str == "" or val_str.lower() in ("null", "nan", "na", "-"):
            val = np.nan
        else:
            try:
                val = float(val_str)
```

```
            except ValueError:
                continue
            if val in sentinels:
                val = np.nan

        try:
            dt = pd.to_datetime(dt_str)
        except Exception:
            continue

        records.append((dt, val))

    if not records:
        raise RuntimeError("Не удалось распарсить данные.")

    s = pd.Series(dict(records)).sort_index()
    s = s[~s.index.duplicated(keep="first")]
    return s

def preprocess(series: pd.Series) -> pd.Series:
    """Заполняет пропуски медианой на регулярной 1-мин сетке."""
    full_idx = pd.date_range(
        series.index.min().floor("min"),
        series.index.max().ceil("min"),
        freq="1min"
    )
    s = series.reindex(full_idx)
    med = float(s.median())
    s = s.fillna(med)
    return s

def extract_features(series: pd.Series, window: int) -> np.ndarray:
    """Формирует матрицу признаков из временного ряда."""
    features = []
    vals = series.values
```

```
    for i in range(len(vals) - window + 1):
        win = vals[i:i+window]
        mean_v = np.mean(win)
        std_v = np.std(win)
        amp_v = np.max(win) - np.min(win)
        features.append([mean_v, std_v, amp_v])
    return np.array(features)


def fuzzy_c_means(data: np.ndarray, c: int, m: float, error: float, maxiter: int):
    """
    Реализация нечеткого алгоритма c-средних.
    data: матрица (N, D)
    Возвращает: centers (c, D), U (c, N)
    """
    N, D = data.shape
    np.random.seed(42)
    U = np.random.rand(c, N)
    U = U / np.sum(U, axis=0)

    for iteration in range(maxiter):
        U_old = U.copy()

        Um = U ** m
        centers = (Um @ data) / np.sum(Um, axis=1, keepdims=True)

        dist = np.zeros((c, N))
        for i in range(c):
            dist[i] = np.linalg.norm(data - centers[i], axis=1)

        for i in range(c):
            for j in range(N):
                if dist[i, j] == 0:
                    U[i, j] = 1.0
                else:
                    U[i, j] = 1.0 / np.sum((dist[i, j] / dist[:, j]) ** (2 / (m - 1)))
```

```
        if np.linalg.norm(U - U_old) < error:
            print(f"[i] FCM сошлась за {iteration+1} итераций.")
            break

    return centers, U

def main():
    print("=" * 66)
    print(f" Вариант 7: Станция {STATION} (Южный полюс)")
    print("=" * 66)

    X_list = []
    y_list = []

    all_dates = [(d, 0) for d in CLASS_1_DATES] + [(d, 1) for d in CLASS_2_DATES]

    for date_str, label in all_dates:
        print(f"\n[i] Обработка даты: {date_str} (Класс {label})")
        text = download_data(date_str)
        if not text:
            continue

        series = parse_data(text)
        series = preprocess(series)
        print(f"    Точек после заполнения: {len(series)}")

        features = extract_features(series, WINDOW_SIZE)
        print(f"    Извлечено окон (признаков): {len(features)}")

        X_list.append(features)
        y_list.extend([label] * len(features))

    if not X_list:
        sys.exit("Нет данных для анализа.")
```

```
X = np.vstack(X_list)
y_true = np.array(y_list)

X_mean = np.mean(X, axis=0)
X_std = np.std(X, axis=0)
X_scaled = (X - X_mean) / X_std

print(f"\n[i] Итоговая матрица данных: {X_scaled.shape[0]} наблюдений, {X_scaled.shape[1]} признака")

print("\n[i] Запуск Fuzzy C-Means...")
centers, U = fuzzy_c_means(X_scaled, c=N_CLUSTERS, m=M_FUZZY, error=ERROR, maxiter=MAX_ITER)

pred_labels = np.argmax(U, axis=0)

acc1 = np.mean(pred_labels == y_true)
acc2 = np.mean((1 - pred_labels) == y_true)
accuracy = max(acc1, acc2)
print(f"[i] Точность кластеризации (с учетом перестановки): {accuracy*100:.2f}%")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sc1 = axes[0, 0].scatter(X_scaled[:, 0], X_scaled[:, 1], c=U[0], cmap='coolwarm', alpha=0.6, s=10)
axes[0, 0].scatter(centers[0, 0], centers[0, 1], c='red', marker='x', s=150, linewidths=3, label='Центр 1')
axes[0, 0].scatter(centers[1, 0], centers[1, 1], c='blue', marker='x', s=150, linewidths=3, label='Центр 2')
axes[0, 0].set_title("Признаковое пространство: Среднее vs СКО")
axes[0, 0].set_xlabel("Среднее (норм.)")
axes[0, 0].set_ylabel("СКО (норм.)")
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)
```

```
plt.colorbar(sc1, ax=axes[0, 0], label='Степень принадлежности кластеру 0')

sc2 = axes[0, 1].scatter(X_scaled[:, 0], X_scaled[:, 2], c=U[0], cmap='coolwarm', alpha=0.6, s=10)
axes[0, 1].scatter(centers[0, 0], centers[0, 2], c='red', marker='x', s=150, linewidths=3, label='Центр 1')
axes[0, 1].scatter(centers[1, 0], centers[1, 2], c='blue', marker='x', s=150, linewidths=3, label='Центр 2')
axes[0, 1].set_title("Признаковое пространство: Среднее vs Амплитуда")
axes[0, 1].set_xlabel("Среднее (норм.)")
axes[0, 1].set_ylabel("Амплитуда (норм.)")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)
plt.colorbar(sc2, ax=axes[0, 1], label='Степень принадлежности кластеру 0')

axes[1, 0].hist([pred_labels[y_true == 0], pred_labels[y_true == 1]],
        bins=[-0.5, 0.5, 1.5], rwidth=0.8, color=['green', 'red'],
        label=['Класс 1 (Спокойно)', 'Класс 2 (Буря)'])
axes[1, 0].set_title("Распределение предсказа
```